
LLMWikiResearch

Axel Tang

University of Toronto

Fujifilm

axel.tang@mail.utoronto.ca

axel.tang.mn@fujifilm.com

Abstract

This report evaluates a lightweight LLM Wiki-style retrieval workflow against a baseline query-time retrieval setting on TriviaQA validation data. Two runs were executed on 1,000 examples using the same local model (smollm2:135m): (1) per-example evidence retrieval and (2) corpus retrieval over generated markdown wiki articles. In the updated full run, the wiki-corpus setting achieved higher accuracy than the baseline (EM 3.20%, F1 12.54% vs. EM 1.50%, F1 4.74%), with a substantial latency increase. In follow-up LMStudio experiments using qwen3.5-35b-a3b, the wiki-corpus setting reached EM/F1 of 93.0%/95.31% on 100 questions and 97.4%/98.14% on 500 questions, with average generation latency around 6.86–7.30s per example. These runs reinforce that model capacity and serving stack are dominant factors for this workflow.

Attestation of Teamwork

Personal Usage

1 Description

LLM Wiki is a pattern for building personal knowledge bases using LLMs. The core insight is a departure from how most people use LLMs with documents today.

The Problem with RAG (the current norm)

Most people's experience with LLMs and documents looks like RAG: you upload files, the LLM retrieves relevant chunks at query time, and generates an answer. There is no accumulation — the LLM is rediscovering knowledge from scratch on every question.

In a typical RAG system, every single query starts from zero:

1. You ask a question
2. It searches the raw documents for relevant chunks
3. The LLM reads those chunks and generates an answer
4. Nothing is saved

The next time you ask a related question, the whole process repeats from scratch. No synthesis is retained, no connections are remembered, no contradictions get flagged and resolved over time.

Think of it like a researcher who reads all the relevant papers every morning before answering your question, but has amnesia by the next day. They are competent in the moment, but never actually *learn* anything across sessions.

FujiFilm (Axel)

The LLM Wiki Idea

Instead of retrieving from raw documents at query time, the LLM incrementally builds and maintains a persistent wiki — a structured, interlinked collection of markdown files. When you add a new source, the LLM reads it, extracts key information, and integrates it into the existing wiki — updating entity pages, revising topic summaries, noting contradictions, and strengthening the evolving synthesis.

The wiki is the key artifact: it is persistent and compounding. The cross-references are already there. The contradictions have already been flagged. The wiki keeps getting richer with every source you add and every question you ask.

The LLM Wiki flips the script: the “thinking work” happens at **ingest time**. By query time, the hard work is already done. You are asking questions against a pre-compiled, interconnected body of knowledge rather than raw documents.

The Librarian Analogy

- **RAG** is like a librarian who finds books for you every time you ask.
- **LLM Wiki** is like a librarian who has already read all the books, written summaries, cross-referenced everything, and flagged where books contradict each other — so when you ask, the answer is nearly ready.

The Three-Layer Architecture

Raw Sources Your curated, immutable collection of documents. Articles, papers, images, data files. The LLM reads from them but never modifies them.

The Wiki A directory of LLM-generated markdown files — summaries, entity pages, concept pages, comparisons. The LLM owns this layer entirely.

The Schema A document (e.g. `CLAUDE.md` or `AGENTS.md`) that tells the LLM how the wiki is structured and what workflows to follow, making it a disciplined wiki maintainer rather than a generic chatbot.

Why It Works

The tedious part of maintaining a knowledge base is not the reading or the thinking — it is the bookkeeping. Humans abandon wikis because the maintenance burden grows faster than the value. LLMs do not get bored, do not forget to update a cross-reference, and can touch 15 files in one pass.

Limitations and Debates

Critics from the community point out several real limitations:

- The `index.md` navigation approach breaks at scale (~100–200+ pages)
- Contradiction resolution is under-specified — which source wins when two conflict?
- Ingesting very large documents (e.g. a 400-page book) requires a pre-retrieval step that is essentially RAG anyway
- Querying a large wiki eventually becomes a retrieval problem again

Supporters counter that it works beautifully at personal research scale and that the compounding value is genuinely novel.

Note

Karpathy framed this as an “idea file” you hand to a coding agent (like Claude Code) to implement for your specific use case. It is a *pattern*, not a product.

2 Preliminaries and Problem Formulation

The central goal of this project is to evaluate whether the LLM Wiki pattern — an ingest-time knowledge compilation approach — outperforms standard RAG on question-answering tasks relevant to market research, and to integrate the best-performing approach into Nimbo AI.

Standard RAG systems retrieve document chunks at query time using embedding similarity (e.g. dense retrieval with FAISS or BM25 sparse retrieval), then pass those chunks as context to an LLM to generate an answer. The key weakness is statelessness: no knowledge accumulates between queries.

LLM Wiki addresses this by shifting synthesis to ingest time, maintaining a persistent, interlinked markdown knowledge base that the LLM queries rather than the raw documents. The tradeoff is higher upfront cost per document but lower and more consistent query-time cost.

Key concepts:

- **RAG (Retrieval-Augmented Generation):** A pipeline combining a retriever and a generator LLM, where relevant chunks are fetched per query ?.
- **TriviaQA:** A large-scale reading comprehension dataset of trivia questions paired with evidence documents, used as a standard benchmark for RAG systems ?.
- **Exact Match (EM) / F1:** Standard evaluation metrics for open-domain QA benchmarks.

3 Design

The system consists of three components mirroring the LLM Wiki architecture:

1. **Raw Sources** (/raw/): Immutable input documents. For benchmarking, these are the evidence documents from the TriviaQA rc split.
2. **Wiki Layer** (/wiki/): LLM-generated .md files — one summary page per source document, plus cross-linked entity and concept pages updated on every ingest.
3. **Schema** (schema.md): Instructions to the LLM agent defining page formats, ingest workflow, and query protocols.

Two pipelines are compared:

- **Baseline RAG:** BM25 retrieval over raw documents, top- k chunks passed to the LLM at query time.
- **LLM Wiki:** All documents pre-ingested into the wiki; queries answered by the LLM reading relevant wiki pages identified via index.md.

4 Methodology

4.1 Dataset: TriviaQA

TriviaQA ? is a publicly available benchmark well-suited for evaluating RAG systems. It provides question-answer pairs alongside evidence documents, making it possible to evaluate both retrieval quality and final answer accuracy.

It is accessible via the Hugging Face Datasets library:

```
from datasets import load_dataset
ds = load_dataset("trivia_qa", "rc")
```

Two configurations are used:

- **rc (reading comprehension):** Questions paired with pre-supplied evidence documents. Used for controlled comparison between RAG and LLM Wiki, since both systems operate over the same document set.
- **unfiltered:** Questions without pre-supplied evidence. Used to test open-domain retrieval, more representative of real-world market research scenarios.

4.2 Additional Benchmarks

To provide a more complete evaluation, two additional datasets are included:

- **HotpotQA ?**: Requires multi-hop reasoning across multiple documents — directly tests whether LLM Wiki’s cross-referencing provides an advantage over RAG.
- **Natural Questions ?**: Real Google search queries with Wikipedia-based answers, useful for evaluating generalization.

4.3 Evaluation Metrics

Answers are evaluated using:

- **Exact Match (EM)**: Whether the predicted answer exactly matches the ground truth (after normalization).
- **F1 Score**: Token-level overlap between predicted and ground truth answers.

5 Numerical Experiments

5.1 Experimental Setup

TriviaQA validation data was evaluated on 1,000 questions from a local snapshot (TriviaQA_VA_1000.json). The LLM backbone was held constant across runs to isolate retrieval strategy effects. The two evaluated settings were:

- **Example retrieval**: retrieve from each record’s local evidence/context fields (-retrieval-source example).
- **LLM Wiki-style corpus retrieval**: retrieve from generated markdown pages in a local corpus directory (-retrieval-source corpus).

All key runtime parameters are shown below. After the main run, additional wiki-corpus follow-up experiments compared smollm2:135m, qwen3:4b-q4_K_M, and LMStudio-served qwen3.5-35b-a3b across different evaluation sizes.

Table 1: Hyperparameters used in both benchmark runs

Parameter	Value
Dataset	TriviaQA validation local snapshot (1,000 examples)
Main model	smollm2:135m via Ollama
Follow-up models	qwen3:4b-q4_K_M via Ollama; qwen3.5-35b-a3b via LMStudio
Retriever	BM25
Top- <i>k</i> retrieved passages	3
Request timeout	300s
Retrieval modes compared	example and corpus
Corpus (for LLM Wiki-style run)	generated markdown wiki articles

Table 2: Benchmark results on TriviaQA validation (1,000 questions)

Method	EM (%)	F1 (%)	Avg Retrieval Latency (s)	Avg Generation Latency (s)
Baseline (example retrieval)	1.50	4.74	0.00007	0.272
LLM Wiki-style (corpus retrieval)	3.20	12.54	0.02120	5.513

6 Discussion

TriviaQA’s rc split is particularly well-suited for this comparison because both systems operate over the same fixed evidence documents, isolating the effect of knowledge compilation (LLM Wiki)

Table 3: Follow-up wiki-corpus model comparison

Model / Setting	EM (%)	F1 (%)	Avg Retrieval Latency (s)	Avg Generation Latency (s)
smollm2:135m (20)	0.0	4.69	0.01876	0.786
qwen3:4b-q4_K_M (20)	90.0	92.4	0.01985	8.915
qwen3.5-35b-a3b via LMStudio (100)	93.0	95.31	0.02549	7.297
qwen3.5-35b-a3b via LMStudio (500)	97.4	98.14	0.02603	6.859

versus on-demand retrieval (RAG). The `unfiltered` split provides a harder, more realistic test where retrieval from a larger corpus is required.

In the updated full 1,000-example run, the LLM Wiki-style corpus setting improved over the baseline on EM/F1, but with a large runtime cost. This suggests that wiki retrieval can improve answer quality when the indexed corpus is answer-bearing and retrieval is correctly pointed at the wiki layer.

These results should still be interpreted cautiously: model quality and corpus engineering remain dominant factors, and latency increases significantly under corpus mode. The follow-up model comparison on the same wiki corpus supports this interpretation: stronger models delivered large EM/F1 gains. In particular, LMStudio-served qwen3.5-35b-a3b achieved 93.0%/95.31% on 100 questions and 97.4%/98.14% on 500 questions.

7 Conclusions

This project investigates whether shifting knowledge synthesis from query time to ingest time — as proposed by the LLM Wiki pattern — yields measurable improvements over standard RAG on established benchmarks including TriviaQA, HotpotQA, and Natural Questions. If validated, the approach will be integrated into Nimbo AI to support more reliable and compounding market research workflows. Further work could explore hybrid approaches that combine LLM Wiki’s persistent knowledge layer with vector search for scalability beyond a few hundred documents.

Based on current measured runs, the updated full smollm2 wiki-corpus benchmark outperforms the baseline on EM/F1, but at much higher inference latency. Immediate next steps are to repeat the same comparison with stronger models and larger-scale runs to verify that the gain is robust and not configuration-sensitive.

The practical takeaway is that this version now shows a plausible quality advantage for LLM Wiki-style retrieval in this setup, with a clear quality-latency tradeoff. The updated evidence suggests that, when retrieval targets answer-bearing wiki pages, model capacity and serving setup remain key bottlenecks for correctness and throughput. The LMStudio qwen3.5-35b-a3b runs on 100 and 500 examples provide stronger support than the earlier 20-question slice.

References

- [1] Joshi, M., Choi, E., Weld, D., & Zettlemoyer, L. (2017). TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. *Proceedings of ACL*.
- [2] Karpathy, A. (2026). LLM Wiki. <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>

Appendix

Reproducible benchmark commands used for wiki-corpus evaluation:

A.1 smollm2:135m

```
cd "/Users/axelmacbookair/Desktop/LLMWikiResearch/Work/llm-wiki/benchmarks/triviaqa" && \
python3 -u benchmark_triviaqa.py \
  --data-file data/TriviaQA_VA_1000.json \
  --limit 1000 \
```

```
--retrieval-source corpus \
--corpus-dir /Users/axelmacbookair/Desktop/LLMWikiResearch/Work/llm-wiki/wiki \
--top-k 3 \
--model smollm2:135m \
--request-timeout 300 \
--output results_smollm2_wiki_corpus_limit1000.json
```

Note: this benchmark writes the JSON output file only after the full run finishes.

A.2 qwen3:4b-q4_K_M (4-bit)

```
cd "/Users/axelmacbookair/Desktop/LLMWikiResearch/Work/llm-wiki/benchmarks/triviaqa" && \
python3 -u benchmark_triviaqa.py \
--data-file data/TriviaQA_VA_1000.json \
--limit 20 \
--retrieval-source corpus \
--corpus-dir /Users/axelmacbookair/Desktop/LLMWikiResearch/Work/llm-wiki/wiki \
--top-k 3 \
--model qwen3:4b-q4_K_M \
--request-timeout 300 \
--output results_qwen3_q4km_wiki_corpus_limit20.json
```

A.3 LMSTUDIO (qwen3-model-32B)

```
cd "/Users/axelmacbookair/Desktop/LLMWikiResearch/Work/llm-wiki/benchmarks/triviaqa" && \
python3 -u benchmark_triviaqa.py \
--data-file data/TriviaQA_VA_1000.json \
--limit 100 \
--retrieval-source corpus \
--corpus-dir /Users/axelmacbookair/Desktop/LLMWikiResearch/Work/llm-wiki/wiki \
--top-k 3 \
--provider lmstudio \
--host "http://10.86.30.140:1234" \
--model "qwen3.5-35b-a3b" \
--request-timeout 300 \
--output results_lmstudio_wiki_corpus_limit100.json

cd "/Users/axelmacbookair/Desktop/LLMWikiResearch/Work/llm-wiki/benchmarks/triviaqa" && \
python3 -u benchmark_triviaqa.py \
--data-file data/TriviaQA_VA_1000.json \
--limit 500 \
--retrieval-source corpus \
--corpus-dir /Users/axelmacbookair/Desktop/LLMWikiResearch/Work/llm-wiki/wiki \
--top-k 3 \
--provider lmstudio \
--host "http://10.86.30.140:1234" \
--model "qwen3.5-35b-a3b" \
--request-timeout 300 \
--output results_lmstudio_wiki_corpus_limit500.json
```